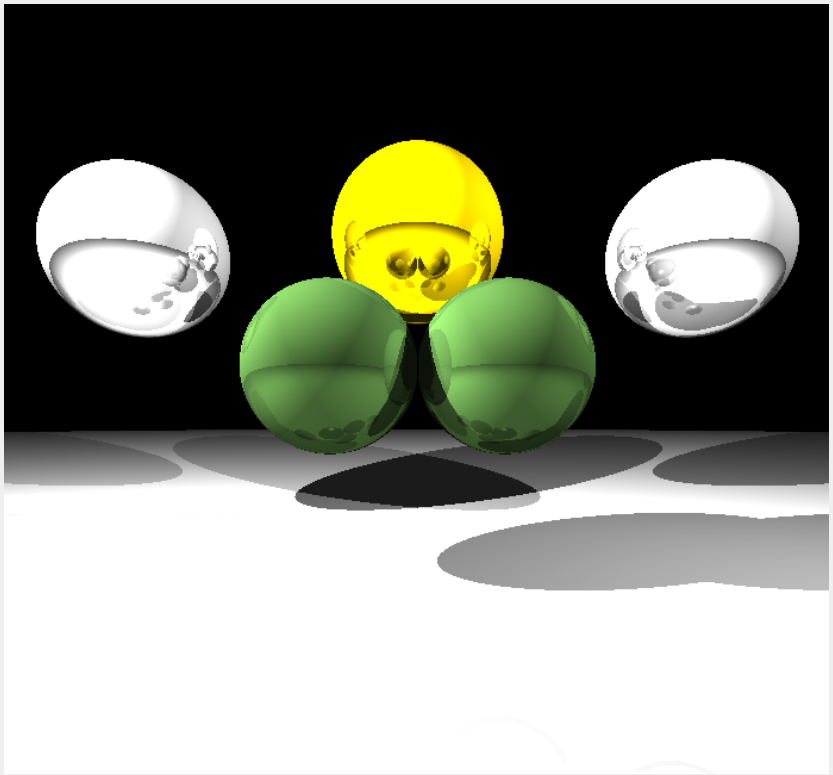
Ray Tracing Workshop

A Hands-On Introduction

Aditya Raj | DEBUG Club



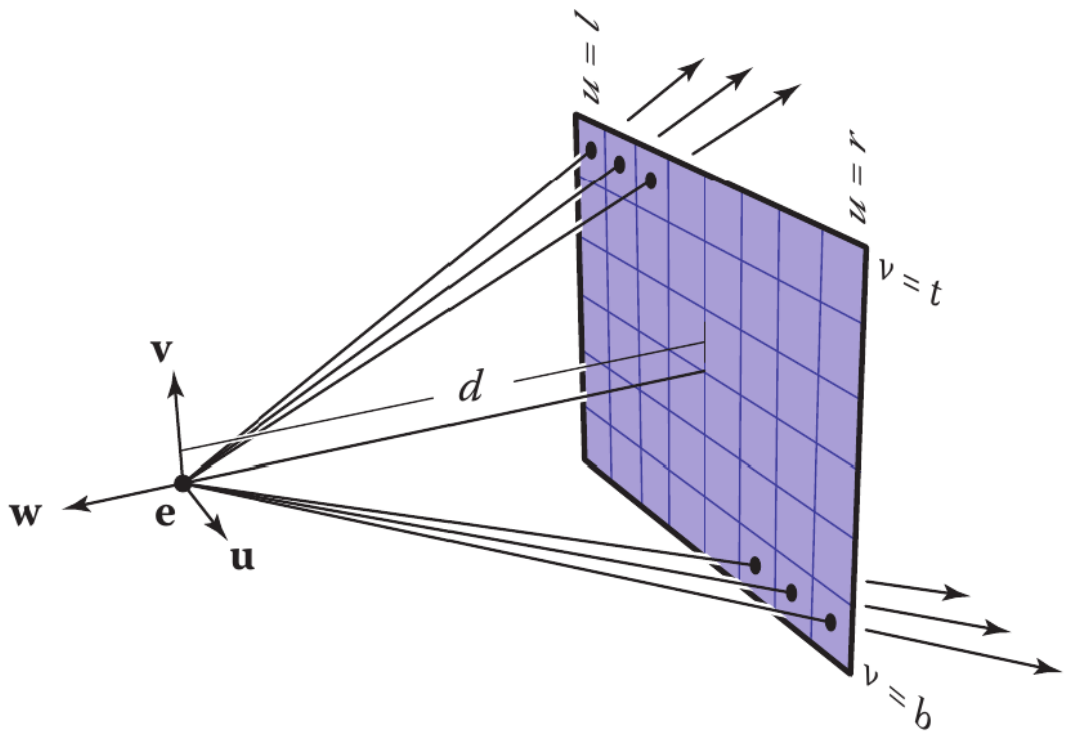
April 2026

What is Rendering?

Rendering takes a scene description as input and produces an image (pixel array) as output.

- Approaches include radiosity, rasterization, and ray tracing.
- Ray tracing is an **image-order rendering** method.
- Each pixel is computed by tracing light-related rays through the scene.

Camera to Viewport Rays



Perspective projection

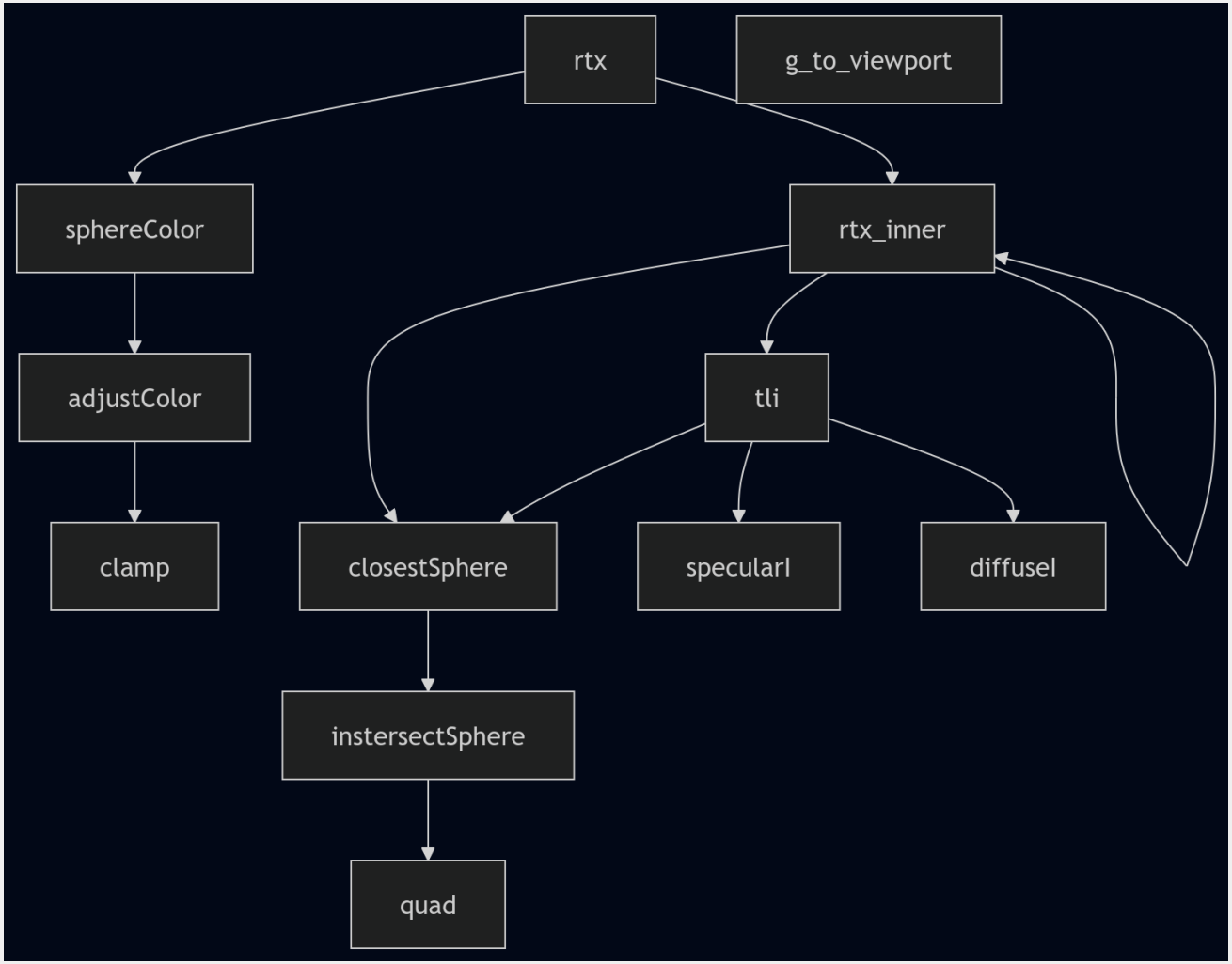
Basic Structure of a Ray Tracer

1. **Ray Generation**: compute viewing rays from camera geometry.
2. **Ray Intersection**: find nearest object hit.
3. **Shading**: compute color from lights, materials, and visibility.

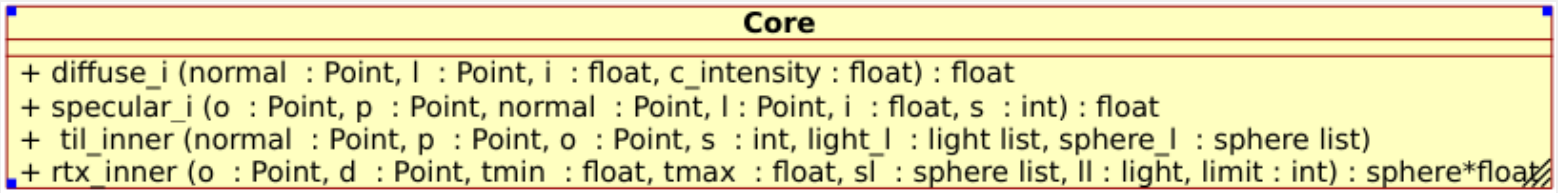
Pseudo-code:

```
for each pixel do
 compute viewing ray
 if ray hits an object then
  compute surface normal
  evaluate shading model
  set pixel color
 else
  set pixel to background color
```

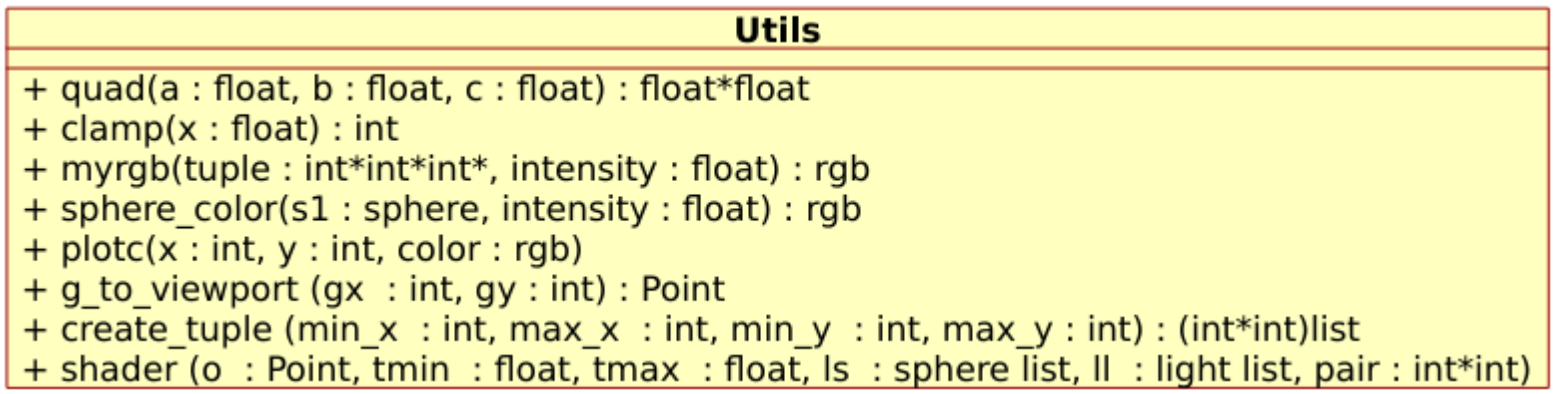
Architecture



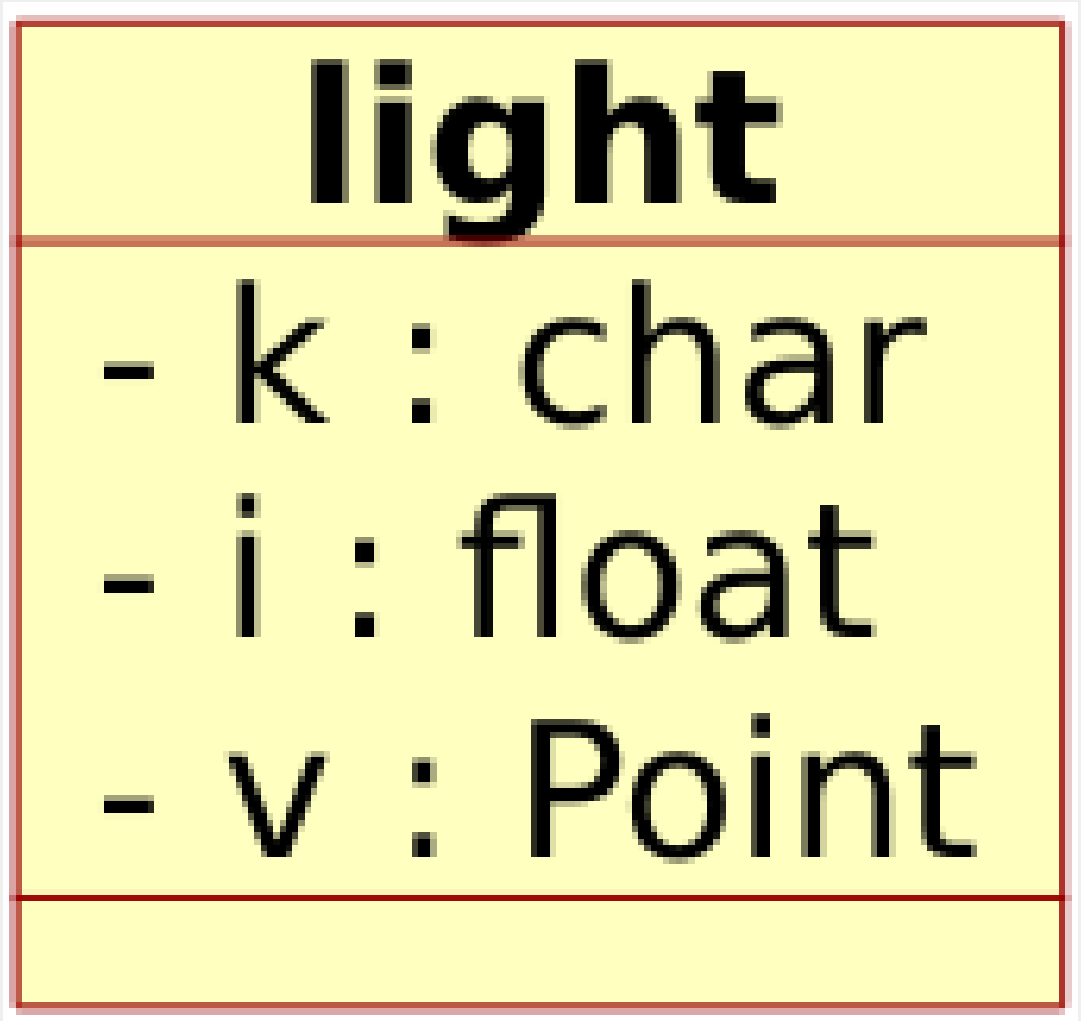
Core Helper Functions



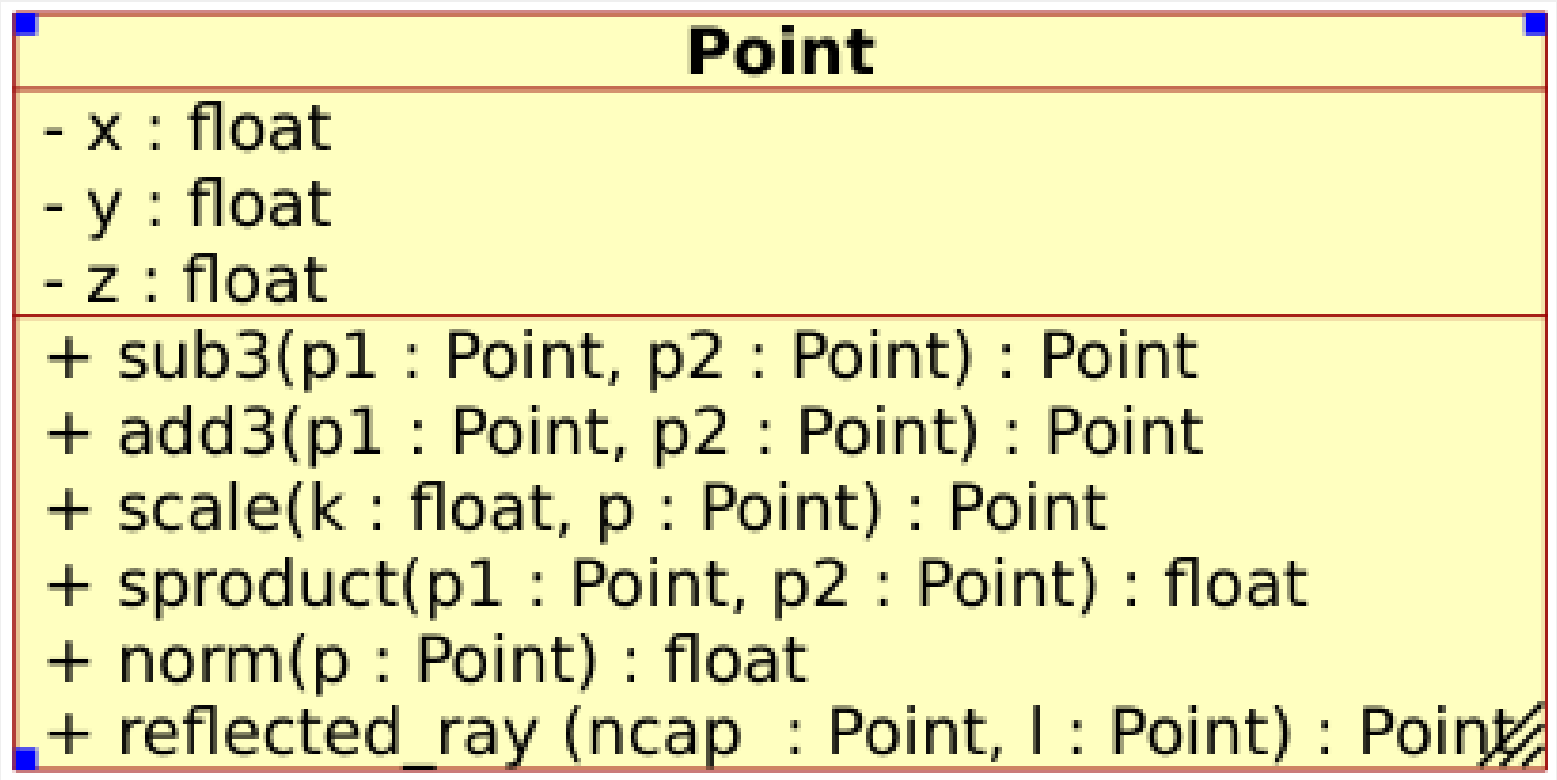
Utility Functions Diagram



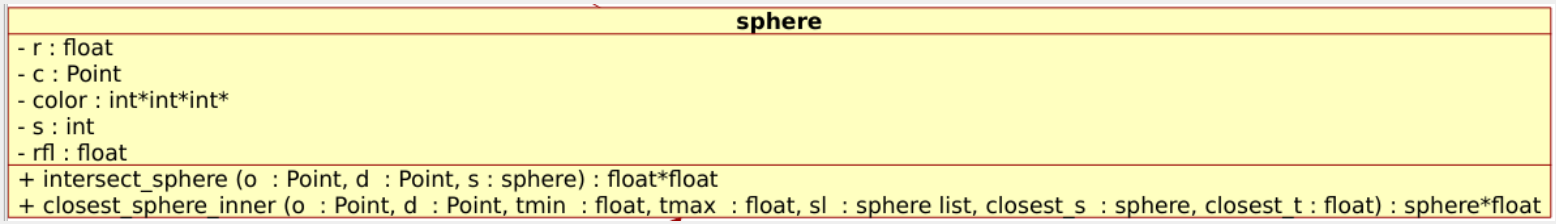
Light Model Diagram



Point Structure Diagram



Sphere Structure Diagram



Color Model (RGB)

- Use additive RGB with channel range $[0, 255]$.
- Treat color as a vector (r, g, b).
- Brightness scaling by k: $k(r, g, b) = (kr, kg, kb)$.
- Clamp each channel to valid bounds after operations.

Scene and Camera Setup

- Camera at origin $O(0, 0, 0)$, looking along positive z.
- Viewport in front of camera at distance d.
- Typical setup: $V_w = V_h = d = 1$ gives FOV $\approx 53°$.
- Pixel-to-viewport mapping:

$$V_x = G_x \frac{V_w}{G_w}, \quad V_y = G_y \frac{V_h}{G_h}, \quad V_z = d$$

Ray Equation

Rays from camera through viewport are modeled as:

$$P = O + t(V - O)$$

- $t < 0$: behind camera
- $0 \leq t \leq 1$: between camera and viewport
- $t > 1$: in front of viewport

Sphere Representation

Sphere equation:

$$\langle P - C, P - C \rangle = r^2$$

Data fields used in code:

- center C, radius r
- surface color
- shininess s
- reflectiveness $rfl \in [0, 1]$

Ray–Sphere Intersection

Substitute ray $P = O + t\vec{D}$ into sphere equation to get:

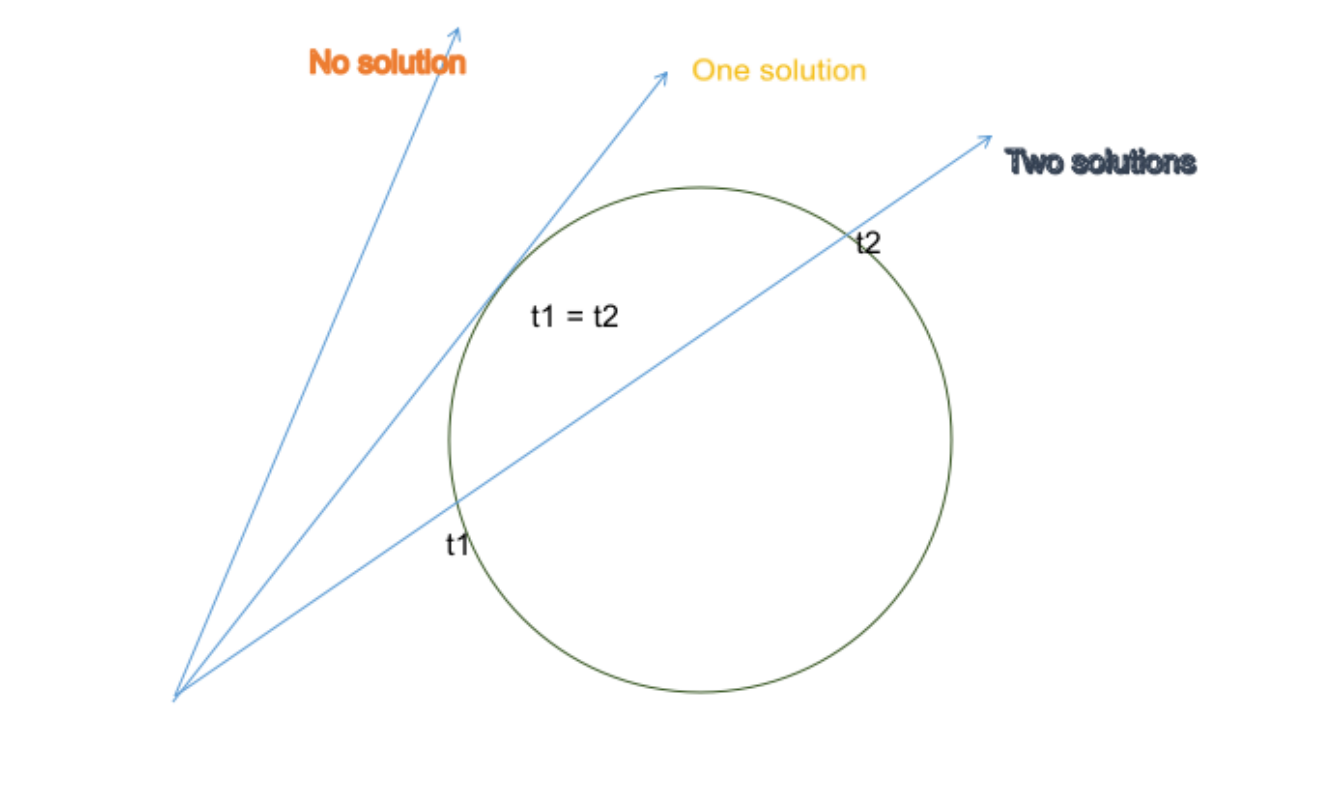
$$at^2 + bt + c = 0$$

where

$$a = \|\vec{D}\|^2, \quad b = 2\langle \vec{D}, \vec{CO} \rangle, \quad c = \|\vec{CO}\|^2 - r^2$$

$$t_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Ray–Sphere Intersection Geometry



Light Types

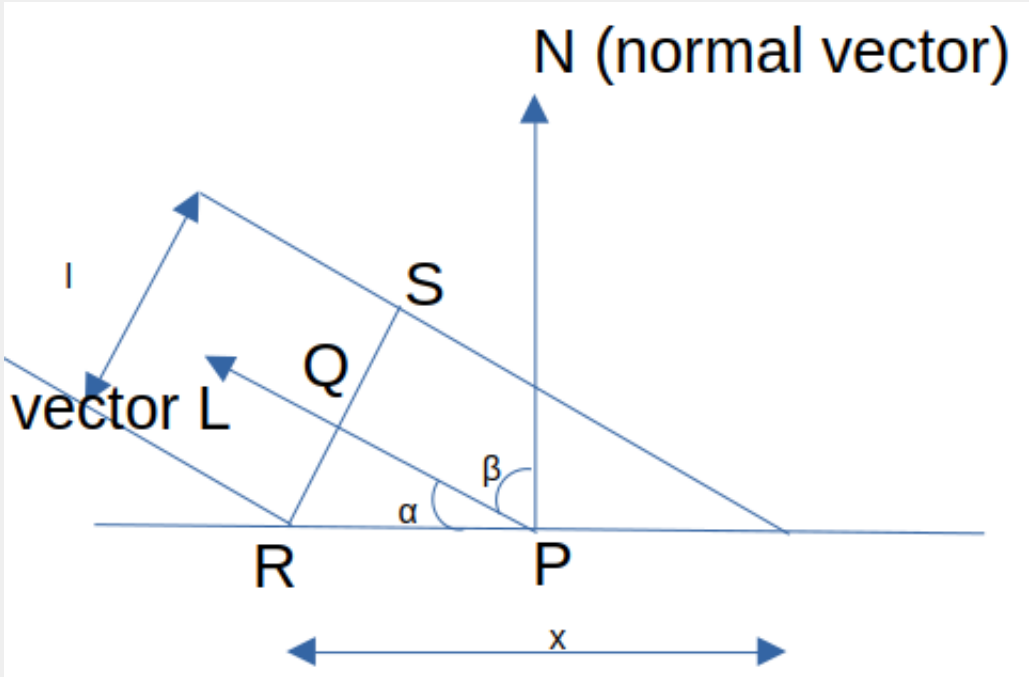
- **Ambient**: constant scene-wide intensity.
- **Point**: position + intensity.
- **Directional**: direction + intensity.

The workshop uses white lights and sums contributions per point.

Diffuse (Lambertian) Reflection

Diffuse term for each non-ambient light:

$$I_{\text{diffuse}} = \sum I_i \max\left(0, \frac{\langle \vec{L}_i, \vec{N} \rangle}{\|\vec{L}_i\| \, \|\vec{N}\|}\right)$$



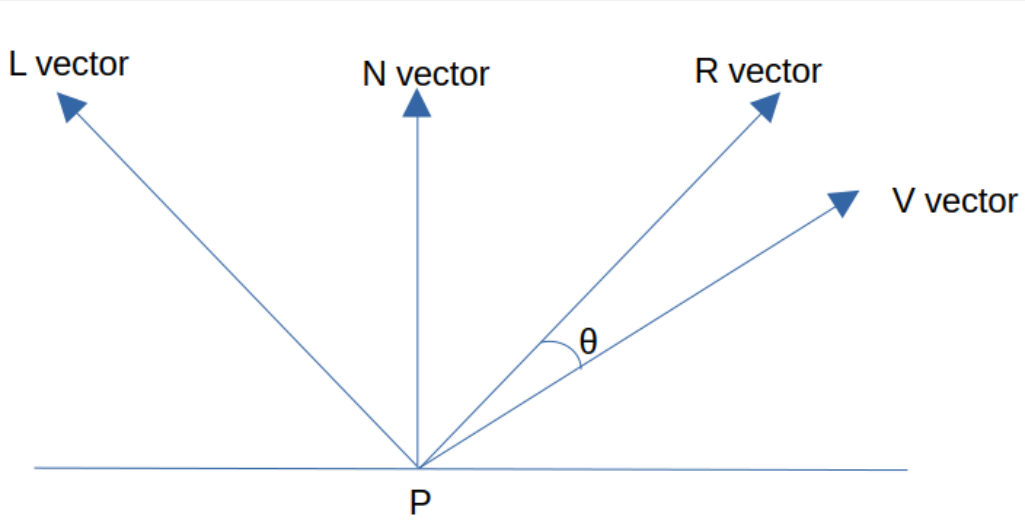
Specular Reflection

Reflection vector:

$$\vec{R} = 2\langle \vec{L}, \hat{N} \rangle \hat{N} - \vec{L}$$

Specular term:

$$I_{\text{specular}} = \sum I_i \max\left(0, \frac{\langle \vec{R}_i, \vec{V} \rangle}{\|\vec{R}_i\| \, \|\vec{V}\|}\right)^s$$



Combined Illumination

$$I_{\text{total}} = I_a + \sum I_i \left[\frac{\langle \vec{L}_i, \vec{N} \rangle}{\|\vec{L}_i\| \, \|\vec{N}\|} + \left(\frac{\langle \vec{R}_i, \vec{V} \rangle}{\|\vec{R}_i\| \, \|\vec{V}\|} \right)^s \right]$$

Contributions are accumulated and applied to object color with channel clamping.

Shadow Rays

To decide if a light contributes at point P, trace a secondary ray:

$$P + t\vec{L}$$

- Point light: check $t \in [\epsilon, 1]$.
- Directional light: check $t \in [\epsilon, \infty)$.
- If blocked by an object, skip diffuse/specular from that light.

Mirror Reflection (Recursive Rays)

- At hit point, compute reflected viewing direction.
- Trace reflected ray recursively.
- Blend local and reflected colors using reflectiveness *rfl*.
- Low recursion depths (e.g., 3) often give good visual quality.

Experiments

1. Change image plane distance d.
2. Replace `double` with `float`.
3. Vary recursion limit in ray tracing.
4. Change viewport dimensions (V_w, V_h).

Compilation Commands

- Plain build:
  `gcc -o render render.c raytracer.c -lm -lSDL3`
- ASan + debug:
  `gcc -g -fsanitize=address -o render3 render3.c raytracer.c -lm -lSDL3`
- OpenMP:
  `gcc -fopenmp -O2 -o render render.c raytracer.c -lm -lSDL3`

Further improvements

1. Add a scene parser code that reads a json file and loads the scene at runtime. (As opposed to hardcoded scene at compile-time.)
2. Replace `double` with `float`.
3. Extend it to other primitives like triangle.
4. Add colored lights, i.e., instead of single intensity, there is different intensity field for each color channel.
5. Add light intensity attenuation.
6. Encapsulate Scene data into single struct.

References

John F. Hughes, Andries van Dam, Morgan McGuire, David F. Sklar, James D. Foley, Steven K. Feiner, and Kurt Akeley.

***Computer Graphics: Principles and Practice*.**

3rd edition, 2013.

Steve Marschner and Peter Shirley.

***Fundamentals of Computer Graphics*.**

4th edition, 2015.

Gabriel Gambetta.

***Computer Graphics from Scratch*.**

No Starch Press, 2021.

Questions?